30. Kimlik Doğrulama ve Yetkilendirme: JWT, OAuth2 ve Session Yönetimi

Authentication (Sen Kimsin?) vs. Authorization (Neye İznin Var?) Ayrımı
Bölüm 5 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"JWT (JSON Web Token) neden 'localStorage' içinde saklanmamalıdır ve bir access token çalındığında sistem nasıl korunur?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Authentication (Kimlik Doğrulama) 'Sen kimsin?' sorusudur; Authorization (Yetkilendirme) ise 'Neye erişebilirsin?' sorusudur. - 📌 JWT'ler şifrelenmiş değil, imzalanmış (signed) jetonlardır; içeriği herkes okuyabilir, bu yüzden içine asla şifre veya kredi kartı yazılmaz. - 📌 JWT'leri XSS saldırılarından korumak için `localStorage` yerine daima `HttpOnly, Secure` çerezlerde (Cookies) saklayın.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir havalimanına girdiğinizi düşünün.

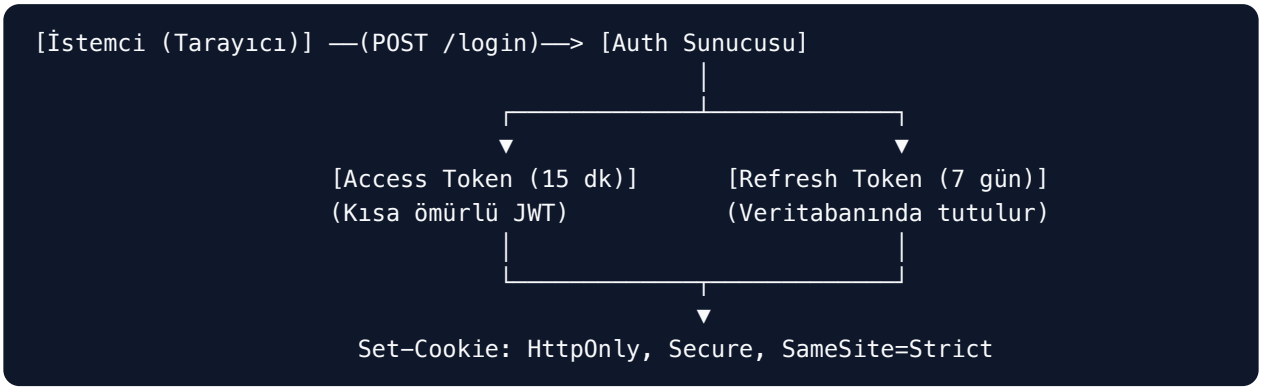
1. **Authentication (Kimlik Doğrulama):** Pasaport kontrolüne gidersiniz. Polis memuru pasaportunuza ve yüzünüze bakar: *"Evet, sen Ali Turgut'sun."* Bu kimlik doğrulamadır.
2. **Authorization (Yetkilendirme):** Uçağa binerken biletinizi gösterirsiniz. Biletiniz 'Ekonomi' ise Business Class bölümüne veya pilot kabinine giremezsiniz! Giriş kartınız (RBAC) sadece koltuk numaranıza izin verir.

Pasaportunuz var diye uçağı uçuramazsınız!

Yazılımda da bir kullanıcının sisteme giriş yapmış olması (AuthN), her veriyi silebileceği (AuthZ) anlamına gelmez. Rolleri ve izinleri (RBAC) sıkı bir kapı nöbetçisi gibi denetlemelisiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Güvenli Auth Mimarisi (HttpOnly Cookie + Refresh Token):



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Web Güvenliği ve Kimlik Yönetimi Uzmanı
Express ve TypeScript kullanan bir API'ye Role-Based Access Control (RBAC)
middleware'i ekleyeceğiz.

## GÖREV:
1. Bir JWT doğrulayan ve kullanıcıyı `req.user` içine koyan `authenticate`
middleware'i yaz.
2. Sadece belirli rollere ('ADMIN', 'SUPERVISOR') izin veren `authorize(...roles)`
middleware'i tasarla.
3. Yetkisiz girişlerde (401 vs 403) doğru HTTP durum kodlarını fırlat.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Güvensiz Yaklaşım: Rolü frontend'den gelen parametreye göre kontrol etmek!
app.delete("/users/:id", async (req, res) => {
  // Saldırgan URL'e ?role=ADMIN yazarak herkesi silebilir!
  if (req.query.role === "ADMIN") {
    await db.user.delete({ where: { id: req.params.id } });
    return res.send("Silindi");
  }
  res.status(403).send("Yetkisiz");
});
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";

export interface UserTokenPayload {
  userId: string;
  email: string;
  role: "USER" | "ADMIN" | "SUPERVISOR";
}

// 1. Kimlik Doğrulama (Authentication – 401)
export function authenticate(req: Request, res: Response, next: NextFunction) {
  const token = req.cookies?.accessToken || req.headers.authorization?.split(" ")
[1];

  if (!token) {
    return res.status(401).json({ error: "Oturum açılmadı (Token eksik)." });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET!) as UserTokenPayload;
    (req as any).user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: "Geçersiz veya süresi dolmuş token." });
  }
}

// 2. Yetkilendirme (Authorization / RBAC – 403)
export function authorize(...allowedRoles: Array<UserTokenPayload["role"]>) {
  return (req: Request, res: Response, next: NextFunction) => {
    const user = (req as any).user as UserTokenPayload;

    if (!user || !allowedRoles.includes(user.role)) {
      return res.status(403).json({ error: "Yetersiz yetki: Bu kaynağa erişim
izniniz yok." });
    }

    next();
  };
}

// 3. Güvenli Rota Kullanımı:
// app.delete("/users/:id", authenticate, authorize("ADMIN"), deleteUserHandler);
```

💡 **Kod Analizi:** `401 Unauthorized` (Kimliğinizi doğrulamadınız, giriş yapın) ile `403 Forbidden` (Kimliğinizi biliyorum ama buraya girmeye yetkiniz yok) ayrımı tam olarak uygulanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** JWT içine kullanıcının şifresini veya kredi kartı numarasını koymak.
  - **Çözüm:** JWT şifrelenmiş değildir (Base64'tür); içine sadece zararsız kullanıcı ID ve rolü koyun.
- **Tuzak:** JWT'yi frontend'de `localStorage` içine kaydedip XSS saldırılarına açık kapı bırakmak.
  - **Çözüm:** Token'ları daima `HttpOnly` çerezlerde saklayarak JavaScript ile okunmasını engelleyin.
- **Tuzak:** 401 ve 403 durum kodlarını birbirine karıştırmak.
  - **Çözüm:** Giriş yapılmadıysa 401, giriş yapılmış ama rol yetersizse 403 dönün.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 30: Güvenli Kimlik Yönetimi - JWT ve RBAC Mimarisi*

- 🖥️ **Slayt 1:** Authentication (Kimlik) vs Authorization (Yetki) farkı.
- 🖥️ **Slayt 2:** JWT'nin anatomisi: Header, Payload, Signature.
- 🖥️ **Slayt 3:** Neden localStorage değil HttpOnly Cookie?
- 🖥️ **Slayt 4:** Bölüm 5 Özeti: Veritabanı, Modelleme ve Backend Ustalığı.

Canlı Kodlama / Ekran Kaydı Senaryosu:
jwt.io sitesini açın. Basit bir JWT oluşturup payload'ın şifresiz şekilde herkes tarafından okunabildiğini gösterin. Ardından Express'te `authorize('ADMIN')` middleware'i kurup normal kullanıcıyla istek atınca 403 Forbidden aldığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki bir rotayı inceleyin. Eğer kullanıcı rolü frontend'den gelen bir parametreye göre kontrol ediliyorsa, bunu JWT içindeki güvenli claims payload'ına göre kontrol edecek şekilde refactor edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir web uygulamasında `401 Unauthorized` ile `403 Forbidden` arasındaki fark nedir? - A) Hiçbir farkı yoktur - B) 401 kullanıcının henüz oturum açmadığını (kimlik doğrulanmadığını); 403 ise kullanıcının oturum açtığını fakat o kaynağa erişim yetkisinin olmadığını belirtir - C) 401 sadece telefonlarda çıkar - D) 403 sunucunun bozulduğunu belirtir > **Doğru Yanıt:** B > **Açıklama:** 401 kimlik eksikliğini (AuthN), 403 ise yetki yetersizliğini (AuthZ) ifade eder.

Soru 2: JWT token'larının `localStorage` yerine `HttpOnly` çerezlerde saklanmasının birincil güvenlik sebebi nedir? - A) HttpOnly çerezlerin CSS'i daha hızlı render etmesi - B) Kötü niyetli JavaScript kodlarının (XSS saldırıları) çereze doğrudan erişip token'ı çalamamasını sağlamak - C) Sadece Chrome tarayıcısında çalışması - D) Token'ın boyutunu küçültmesi > **Doğru Yanıt:** B > **Açıklama:** `HttpOnly` bayrağı, tarayıcıda çalışan JavaScript kodunun (XSS) çerezi okumasını tarayıcı seviyesinde engelleyerek token hırsızlığını önler.